

STUDYIELD + STUDY RPG

FOOLPROOF MASTER IMPLEMENTATION PROMPT V2

You are Codex operating through Goose CLI as the lead software architect, senior full-stack engineer, database engineer, RAG engineer, security reviewer, game-system designer, test engineer and deployment engineer for a major rebuild of Studyield.

You are not creating a quick prototype.

You are building a maintainable, efficient and secure student platform that combines:

1. **Studyield** — practical tools that help students study.
2. **Study RPG** — a complete study-powered RPG, card game and controlled card economy.

The project is operated by **Nightmare Studio**.

The first intended users are school students, beginning with CBSE Grade 9 students in India, but the architecture must support other grades, boards, schools, countries and languages.

The frontend will be hosted on **Cloudflare Pages**.

The backend and its persistent supporting services will be hosted through **Enzonic Cloud at cloud.enzonic.com** or an equivalent deployment documented by this project.

The project must remain usable without real-money payments, advertisements or mandatory paid AI APIs.

However:

Do not weaken the architecture merely to minimise hosting cost.

Redis, queues, WebSockets and production-grade RAG are important parts of the platform and must remain.

1. ABSOLUTE OPERATING RULES

Follow these rules throughout the entire implementation.

1.1 Inspect Before Editing

Before changing code:

1. Inspect the entire repository structure.
2. Read both frontend and backend package manifests.
3. Read Docker and deployment files.
4. Inspect existing database migrations.
5. Inspect every backend module.
6. Inspect frontend routes and API clients.
7. Find broken, unfinished, duplicated and placeholder features.
8. Run the existing builds and tests.
9. Record current failures before modifying anything.
10. Create an audit report at:

```
docs/audits/INITIAL_REPOSITORY_AUDIT.md
```

Do not assume the README accurately represents the code.

Do not assume an existing feature works merely because a route or module exists.

1.2 Do Not Blindly Rewrite the Repository

Preserve useful Studyield code.

Rewrite a module only when one or more of these are true:

- It does not build.
- It is fundamentally insecure.
- It is mostly placeholder code.
- It conflicts with the new architecture.
- Repairing it would be more complex than replacing it.
- It has no tests and cannot be understood safely.
- It depends entirely on removed commercial functionality.

For every major rewrite, record the reason in an Architecture Decision Record.

Store decisions under:

```
docs/architecture/decisions/
```

1.3 No Fake Completion

Never claim a feature is complete when:

- The page is only a visual mock-up.

- Buttons do nothing.
- Data exists only in local component state.
- An endpoint returns hard-coded success.
- A test was not run.
- An AI response is faked.
- A database operation is not persisted.
- A battle outcome is calculated only by the client.
- A queue exists but no worker processes it.
- A deployment file has not been validated.

No core page may contain “Coming soon” at the end of the implementation.

1.4 Keep the Application Buildable

After every major phase, run:

- Install
- Lint
- Type checking
- Unit tests
- Integration tests
- Frontend build
- Backend build

Do not continue piling changes onto an already broken branch.

1.5 Protect Existing Work

Before destructive changes:

- Create a git checkpoint.
- Record the current commit.
- Preserve existing migrations.
- Never rewrite production migration history.
- Never delete user data without a migration and rollback plan.
- Never run destructive database commands automatically against an unknown environment.

1.6 No Unnecessary Microservices

Build a **modular monolith with separate worker processes**.

Do not split every feature into independent network services.

The recommended deployment consists of:

- Frontend
- API process

- Worker process
- PostgreSQL
- Redis
- Qdrant
- Object storage

These are separate runtime components, but the API and worker share the same codebase, domain modules and database contracts.

2. AGENT WORKFLOW USING GOOSE AND CODEX

Create durable agent instructions so work can continue across sessions.

Required files:

- AGENTS.md
- IMPLEMENTATION_STATUS.md
- docs/architecture/SYSTEM_MAP.md
- docs/audits/INITIAL_REPOSITORY_AUDIT.md
- .goose/recipes/
- .goose/prompts/

Before generating Goose configuration, inspect the installed Goose version and its current recipe schema.

Do not guess obsolete recipe syntax.

Create Goose recipes for:

- Repository audit
- Architecture stabilisation
- Database and migrations
- Studyield tools
- RAG and document ingestion
- Study RPG battle engine
- Card economy
- Events and StudyPass
- Security review
- Test and build verification
- Deployment verification

Use Goose subagents where supported.

Suggested subagent responsibilities:

- Repository Auditor

- Database Engineer
- Redis and Queue Engineer
- RAG Engineer
- Frontend Engineer
- Study Tools Engineer
- Game Engine Engineer
- Economy Engineer
- Security Reviewer
- Test Engineer
- Deployment Reviewer

Subagents must write findings into repository documents rather than leaving essential findings only in chat context.

Codex must remain the primary implementation agent.

Goose should coordinate work, preserve context and launch specialised reviews.

Do not allow multiple agents to edit the same file simultaneously.

3. LICENCE AUDIT — DO NOT GUESS

The repository currently appears to contain conflicting licence declarations.

The root licence file and package or README metadata may not agree.

Perform a licence audit before branding or redistribution changes.

Create:

```
docs/audits/LICENSE_AUDIT.md
```

The audit must identify:

- Root licence file
- Package licence fields
- README licence statements
- File-level copyright notices
- Third-party copied assets
- Fonts
- Icons
- illustrations
- AI models
- datasets
- question banks

- translations

Until the contradiction is resolved:

- Preserve all upstream copyright notices.
- Preserve the root licence.
- Do not replace it with Apache, MIT or another licence.
- Do not claim Nightmare Studio wrote the upstream project.
- Mark modified files appropriately.
- Add `UPSTREAM.md`.
- Add `THIRD_PARTY_NOTICES.md`.
- Add a visible Source Code and Licence section in the application.
- Do not provide legal conclusions without evidence.

All Study RPG characters, card names, lore and artwork must be original.

Do not use copyrighted characters or artwork from The Amazing Digital Circus.

“Abstraction” may remain a general conceptual inspiration, but the implementation must have original terminology, writing and visual design.

4. REQUIRED TECHNICAL ARCHITECTURE

4.1 Frontend

Retain and improve the existing frontend where practical.

Preferred stack:

- React
- TypeScript
- Vite
- React Router
- TanStack Query
- Zustand
- Tailwind CSS
- Radix UI where already used
- i18next
- Zod or equivalent runtime validation
- Workbox or a suitable Vite PWA integration
- Vitest
- React Testing Library
- Playwright

Do not migrate to a different frontend framework without an evidence-based reason.

Use route-level lazy loading.

Split the Studyfield and Study RPG route bundles.

Optimise for inexpensive Android phones and slower school computers.

4.2 Backend

Retain NestJS and TypeScript unless the audit proves the backend is beyond repair.

Use:

- NestJS modular monolith
- REST API
- Socket.IO or standards-compatible WebSockets
- PostgreSQL
- Redis
- BullMQ
- Qdrant
- S3-compatible storage adapter
- OpenAPI
- Class validation or an equivalent validated DTO system
- Structured logging
- Jest or the existing backend test system

Do not perform a major NestJS upgrade during the rebuild unless:

- Existing dependencies require it.
- Security support requires it.
- Migration documentation has been reviewed.
- Tests prove compatibility.

4.3 Source-of-Truth Rule

PostgreSQL is the authoritative source of truth.

PostgreSQL must own:

- Users
- Authentication sessions
- Study data
- Questions
- Quizzes
- Tasks
- Exams
- Player progression

- Currency balances
- Wallet ledger
- Cards
- Ownership
- Supply
- Trades
- Battles
- Events
- Quests
- Rewards
- Audit logs
- RAG document metadata
- Ingestion state
- Job audit records

Redis must never be the sole authoritative location for irreplaceable user or economy data.

Qdrant must never be the sole authoritative location for uploaded document metadata.

4.4 Redis Is Required

Redis is a first-class production dependency.

Use Redis for:

- Application caching
- Distributed rate limiting
- BullMQ queues
- Delayed and scheduled jobs
- Short-lived idempotency records
- WebSocket coordination
- Presence
- Matchmaking queues
- Distributed locks
- Leaderboard mirrors
- Temporary battle state
- Quiz question cooldowns
- Anti-abuse counters
- Notification fan-out
- RAG query cache
- Embedding cache references

Do not store permanent wallet balances, card ownership or final rewards only in Redis.

Use namespaced keys such as:

- `studyield:cache:`

- studyield:rate:
- studyield:queue:
- studyield:lock:
- studyield:battle:
- studyield:leaderboard:
- studyield:rag:
- studyield:session:

Add environment and version prefixes.

Example:

```
studyield:prod:v1:cache:card-definition:{uuid}
```

Apply:

- Explicit TTLs
- TTL jitter
- Bounded cache sizes
- Eviction policy documentation
- Connection timeout
- Retry policy
- Circuit breaker behaviour
- Health checks
- Metrics
- Authentication
- TLS where supported

Enable appropriate Redis persistence for the chosen deployment, but still treat PostgreSQL as authoritative.

Document RDB and AOF configuration.

4.5 BullMQ Is Required

Use BullMQ for work that should not block HTTP requests.

Queues should include:

- Document parsing
- OCR when required
- Document chunking
- Embedding generation
- Qdrant indexing
- RAG reindexing
- AI generation
- Email delivery
- Web push

- Analytics aggregation
- Study streak processing
- Event activation
- StudyPass ending
- Burn instalment payments
- Card extinction checks
- Replacement card activation
- Expired marketplace listings
- Pending reward expiry warnings
- Account export
- Account deletion
- File cleanup
- Backups
- Scheduled notifications

Every job must have:

- Stable job type
- Schema-validated payload
- Explicit timeout
- Retry count
- Exponential backoff where appropriate
- Unique or deterministic job ID
- Idempotency protection
- Failure logging
- Dead-letter or failed-job inspection
- Maximum retention
- Admin visibility
- Correlation ID

Workers must not perform a large multi-step operation without recording progress.

Use small atomic jobs or BullMQ flows.

4.6 Transactional Outbox

Implement the transactional outbox pattern.

When a PostgreSQL transaction creates an event that requires background work:

1. Write the domain change.
2. Write an outbox event in the same PostgreSQL transaction.
3. Commit.
4. An outbox dispatcher enqueues the event into BullMQ.
5. Mark the outbox event dispatched.
6. The worker processes it idempotently.
7. Store processing status.

This is required for:

- Rewards
- Card burns
- Card extinction
- Marketplace settlement
- RAG ingestion
- Notifications
- Exam rewards
- Event transitions
- StudyPass rewards

Do not write to PostgreSQL and Redis as two unrelated operations and assume both succeed.

4.7 Qdrant Is Required for Full RAG

Qdrant is the production vector database.

Do not downgrade RAG to simple keyword search.

PostgreSQL full-text search may remain as:

- A fallback
- An exact search tool
- A debugging tool
- A lexical retrieval source

Full RAG must use Qdrant.

4.8 ClickHouse Decision

Do not automatically remove ClickHouse.

Do not automatically require it either.

Place analytics behind an `AnalyticsStore` interface.

Support:

- PostgreSQL analytics implementation
- Optional ClickHouse implementation

Default to PostgreSQL aggregates during the first stable release.

Enable ClickHouse only when benchmarks show a real need, such as:

- Analytics tables becoming too large
- Aggregation jobs affecting core database performance
- Dashboard query latency remaining unacceptable after indexing and pre-aggregation
- Event ingestion reaching a measured volume that PostgreSQL cannot handle comfortably

Create an ADR explaining the decision.

Do not run ClickHouse merely because the upstream project included it.

4.9 Keep Useful S3 Support

Retain an S3-compatible storage abstraction.

Support:

- Cloudflare R2
- MinIO
- Local development filesystem
- Other S3-compatible services

The AWS SDK may remain because R2 and MinIO use S3-compatible APIs.

Do not tie storage directly to AWS.

4.10 Remove or Isolate Unnecessary Systems

The following should not be core requirements:

- Stripe
- Real-money subscriptions
- Firebase
- AWS SES
- Paid-only search providers
- Paid-only code sandboxes
- Apple login
- Google login
- Commercial analytics

Handle them as follows:

Stripe

Remove user-facing subscription and payment functionality.

Study RPG contains no real-money purchases.

Firestore

Replace required Firestore notifications with:

- In-app notifications
- Optional standards-based Web Push using VAPID

Email

Use a generic SMTP adapter.

Email must remain optional during local development.

OAuth

Keep behind feature flags.

Username and password authentication must work without Google or Apple.

Code Sandbox

Disable server-side arbitrary-code execution by default.

Only enable it after isolation, resource limits and security review exist.

5. REPOSITORY STRUCTURE

Keep the repository understandable.

A suitable structure is:

```
/
  frontend/
  backend/
    src/
      common/
      config/
      modules/
      workers/
      jobs/
      migrations/
  packages/
    contracts/
    game-engine/
```

```
shared-types/  
validation/  
docs/  
  architecture/  
  audits/  
  deployment/  
  game/  
  runbooks/  
scripts/  
tests/  
.goose/  
AGENTS.md  
IMPLEMENTATION_STATUS.md
```

Do not introduce Nx, Turborepo or another build orchestrator unless it solves a measured problem.

A simple workspace setup is preferred.

Shared packages must not import framework-specific application code.

The game engine should be a deterministic TypeScript package usable by:

- Backend
- Tests
- Battle simulations
- Admin balance tools

The frontend may use shared types, but never the server's secret battle or loot resolution logic.

6. STUDENT-FIRST PRODUCT PRINCIPLES

Studyield must be genuinely useful before the game layer is considered.

The product must:

- Reduce study friction.
- Encourage active recall.
- Encourage spaced repetition.
- Help students plan.
- Help students find knowledge gaps.
- Provide understandable explanations.
- Work on mobile devices.
- Work with limited internet.
- Avoid shaming.
- Avoid addictive dark patterns.

- Avoid punishment for low marks.
- Avoid encouraging sleep deprivation.
- Avoid surveillance-heavy anti-cheat.
- Clearly identify AI-generated content.
- Allow students to work without AI.
- Protect private uploaded materials.
- Avoid sharing real names publicly.

The game rewards meaningful learning.

The learning tools must never exist merely as a method to farm game currency.

7. STUDYIELD CORE TOOLS

Implement every tool as a functional system rather than a page mock-up.

7.1 Academic Structure

Support:

- Country
- School board
- School
- Grade
- Academic year
- Subject
- Programme
- Chapter
- Topic
- Learning objective
- Exam
- Exam portion

Seed an editable CBSE Grade 9 preset.

Subjects may include:

- Mathematics
- Science
- Social Science
- English
- Hindi
- Malayalam
- Information Technology
- Kaushal Vikas

- Physical Education
- Custom subjects

Do not hard-code these subjects into core game logic.

7.2 Student Dashboard

Show:

- Today's study plan
- Tasks due
- Upcoming exams
- Current exam portions
- Focus time
- Flashcards due
- Revision queue
- Weak topics
- Recent mistakes
- Quiz accuracy
- Puzzle streaks
- Study streak
- STP earned today
- Player XP
- Current Event EXP
- Daily quests
- Recommended next study action

Allow students to hide game statistics and use Studyield as a normal study app.

7.3 Planner and Tasks

Support:

- Homework
- Revision
- Exam preparation
- Projects
- Reading
- Practice
- Recurring tasks
- Subtasks
- Priority
- Due date
- Estimated time
- Actual time
- Subject
- Chapter

- Topic
- Notes
- Evidence
- Completion history

Prevent currency farming through meaningless tasks.

Use:

- Minimum meaningful duration
- Daily reward limits
- Diminishing returns
- Duplicate-task detection
- Server timestamps

Task completion must remain possible without claiming game rewards.

7.4 Focus Sessions

Support:

- Pomodoro
- Custom timer
- Stopwatch
- Break timer
- Session goal
- Subject
- Topic
- Distraction notes
- Pause
- Resume
- Session reflection

Allow offline focus sessions with later synchronisation.

Detect impossible or corrupted durations without accusing the user.

7.5 Notes

Support:

- Markdown
- Rich text where already implemented
- Mathematics
- Images
- Tables
- Checklists

- Tags
- Subjects
- Chapters
- Topics
- Links
- Autosave
- Version history
- Search
- Export
- Print
- Offline editing
- Conflict resolution

7.6 Flashcards

Support:

- Standard cards
- Cloze cards
- Image cards
- Formula cards
- Decks
- Tags
- Imports
- Exports
- Review queue
- Leech detection
- Review history
- Accuracy
- Spaced repetition

Use a tested scheduling algorithm.

Do not invent an unverified scheduling formula.

AI-generated cards must be editable drafts.

7.7 Quiz Engine

Question types:

- Single choice
- Multiple choice
- True or false
- Short answer
- Numerical
- Fill in the blank

- Matching
- Ordering

Every question must include:

- UUID
- Subject
- Chapter
- Topic
- Difficulty
- Answer
- Explanation
- Source
- Verification status
- Author
- Version

Do not send hidden correct answers to the browser before submission.

Support:

- Practice
- Timed quiz
- Exam mode
- Battle quiz
- Daily quiz
- Teacher-approved quiz
- Adaptive quiz

7.8 Mistake Notebook

Add a dedicated mistake notebook.

Automatically record:

- Incorrect quiz answers
- Incorrect puzzle answers
- Flashcards repeatedly failed
- Exam mistakes
- Problem-solving misconceptions
- Battle quiz mistakes

Allow the student to add:

- Correct method
- Reason for mistake
- Reminder
- Follow-up date

Generate a revision queue from mistakes.

7.9 Subject Puzzles

Support original puzzles for:

- Mathematics
- Science
- Languages
- Social Science
- IT
- Logical reasoning
- Custom subjects

Each subject has an independent puzzle streak.

Rules:

- Correct ranked puzzle: streak increases.
- Incorrect ranked puzzle: streak resets unless a valid streak shield is used.
- Practice puzzles do not affect ranked streaks.
- Only a limited number of daily ranked puzzles produce game rewards.
- Show personal best.
- Store full attempt history.
- Never reuse the same ranked puzzle immediately.

7.10 Exam Centre

Support:

- Exam calendar
- Exam portions
- Mark schemes
- Past papers
- Practice papers
- Timers
- Exam results
- Mistake analysis
- Revision plans
- Reward claims

During an exam period:

- Battle quizzes prioritise exam portions.
- Damage challenges prioritise exam portions.
- Daily quests prioritise relevant subjects.
- Revision recommendations prioritise the nearest exam.

Return to normal subject selection after the period ends.

7.11 Exam Clone

Allow users to upload or manually enter past papers.

Analyse:

- Question types
- Marks
- Sections
- Chapter distribution
- Difficulty
- Command words
- Style
- Time allocation

Generate practice papers using:

- Manual templates
- Verified question banks
- Optional RAG
- Optional AI

Generated questions must be labelled and reviewable.

7.12 Problem Solver

Use a learning-first progression:

1. Restate the problem.
2. Identify the concept.
3. Ask what the student has tried.
4. Give a small hint.
5. Give a larger hint.
6. Show one step.
7. Show the complete solution.
8. Generate a similar problem.
9. Record the misconception.

Do not reveal the final answer immediately by default.

7.13 Teach-Back

Allow students to explain concepts through text or transcribed voice.

Evaluate:

- Correct concepts
- Missing concepts
- Misconceptions
- Clarity
- Examples
- Connections
- Confidence

Provide a non-AI checklist mode.

7.14 Formula and Concept Tools

Provide:

- Formula cards
- Formula sheets
- Concept maps
- Chapter summaries
- Glossaries
- Timelines
- Comparison tables
- Revision sheets

All generated content must be editable.

7.15 Learning Paths

A learning path contains:

- Objective
- Prerequisites
- Topics
- Notes
- Flashcards
- Quizzes
- Puzzles
- Practice
- Teach-back
- Mastery check

Learning paths may be manual or AI-assisted.

7.16 Analytics

Show useful, non-shaming analytics:

- Time studied
- Accuracy
- Mastery
- Revision completion
- Flashcards reviewed
- Weak topics
- Improvement
- Puzzle streaks
- Exam performance
- Study consistency
- Game progression
- Reward sources

Avoid meaningless vanity graphs.

8. PRODUCTION RAG SYSTEM

RAG is a core Studyield feature.

It must be reliable, private and citation-grounded.

8.1 RAG Components

Implement:

- Upload service
- File validation
- Content hashing
- Parser
- Chunker
- Embedding provider
- Qdrant index
- Hybrid retriever
- Reranker
- Context builder
- Generation provider
- Citation builder
- Evaluation suite
- Deletion pipeline
- Reindexing pipeline

8.2 Supported Materials

Support:

- PDF
- DOCX
- TXT
- Markdown
- Images where OCR is available
- Manually pasted text
- Notes
- Question papers

Do not repeatedly OCR text-based PDFs.

Detect whether extraction already produced usable text.

8.3 Ingestion State Machine

Use explicit states:

- Uploaded
- Validating
- Rejected
- Queued
- Parsing
- Parsed
- Chunking
- Embedding
- Indexing
- Ready
- Partially Ready
- Failed
- Deleting
- Deleted

Store failures and retry history.

Users must see ingestion progress.

8.4 Deduplication

Calculate a SHA-256 content hash.

Use it to:

- Detect duplicate uploads
- Reuse parsing where allowed
- Reuse embeddings where allowed
- Avoid duplicate Qdrant points
- Prevent repeated ingestion charges
- Detect changed documents

Never allow one user to access another user's material merely because the content hash matches.

8.5 Chunking

Use structure-aware chunking.

Preserve:

- Document title
- Heading hierarchy
- Page number
- Section number
- Paragraph boundaries
- List boundaries
- Table context
- Formula context
- Question numbers

Use configurable chunk size and overlap.

Do not split every document using one fixed character count.

Store parent sections and child chunks.

8.6 Qdrant Collection Design

Do not create one Qdrant collection per user.

Use a small number of versioned collections based on:

- Embedding model
- Vector dimensions
- Content type
- Schema version

Example:

- `study_material_chunks_v1`

- `verified_questions_v1`
- `concepts_v1`

Each point must contain indexed metadata such as:

- Tenant ID
- User ID
- Classroom ID
- Material ID
- Subject ID
- Grade ID
- Board ID
- Chapter ID
- Topic ID
- Language
- Page
- Section
- Verification status
- Visibility
- Content hash
- Embedding version

Create payload indexes before large ingestion.

Every search must apply authorisation filters before returning results.

8.7 Hybrid Retrieval

Implement hybrid retrieval using:

- Dense semantic retrieval
- Sparse or lexical retrieval
- Metadata filters
- Rank fusion
- Optional reranking

Default retrieval flow:

1. Validate user and material access.
2. Normalise the query.
3. Detect subject and topic where possible.
4. Produce dense query embedding.
5. Produce lexical or sparse query.
6. Apply mandatory ownership and visibility filters.
7. Retrieve candidate chunks.
8. Fuse candidate rankings.
9. Remove near-duplicates.
10. Prefer source diversity.

11. Rerank the strongest candidates.
12. Apply minimum relevance threshold.
13. Build a token-bounded context.
14. Generate an answer.
15. Attach citations.
16. Record retrieval diagnostics without storing sensitive answer content unnecessarily.

8.8 Grounding Rules

The assistant must:

- Cite the material.
- Cite page or section when available.
- Distinguish source content from its own explanation.
- State when evidence is insufficient.
- Avoid inventing page numbers.
- Avoid claiming the source says something not present.
- Never use chunks from another user.
- Never answer from inaccessible materials.
- Never treat AI-generated notes as verified textbook facts automatically.

Provide modes:

- Strict source-only
- Source-first
- General explanation
- Quiz me
- Socratic
- Exam revision
- Find exact passage

8.9 Embedding Providers

Create an `EmbeddingProvider` interface.

Support adapters for:

- Qdrant Cloud inference where configured
- Local embedding service
- OpenAI-compatible embeddings
- Other administrator-configured providers

Do not hard-code one paid model.

Store embedding model and version with every indexed point.

When the embedding model changes:

- Create a new collection version.
- Reindex in the background.
- Keep the old index available until the new index is ready.
- Switch through a feature flag.
- Roll back if evaluation worsens.

8.10 Retrieval Evaluation

Create a test dataset containing:

- Questions
- Expected source documents
- Expected pages or sections
- Relevant chunks
- Irrelevant distractors

Measure:

- Recall at K
- Precision at K
- Citation correctness
- Unsupported-claim rate
- Cross-user leakage
- Latency
- Empty-result handling

RAG is not complete merely because Qdrant returns vectors.

9. AI PROVIDER ARCHITECTURE

Create provider interfaces for:

- Chat
- Structured generation
- Vision
- Embeddings
- Speech transcription
- Reranking

Support:

- OpenRouter
- OpenAI-compatible endpoints
- Local Ollama

- Administrator-provided providers
- Disabled AI mode

Never expose API keys to the frontend.

Implement:

- Provider timeout
- Retries
- Circuit breaker
- Daily quotas
- Request budgets
- Model capability detection
- Structured-output validation
- Streaming
- Cancellation
- Fallback
- Response caching where safe
- Content provenance

No essential study tool may become unusable because AI is unavailable.

10. STUDY RPG IDENTITY

Study RPG is the game layer connected to Studyield.

Lore concept:

Knowledge generates Mana.

Distraction, unfinished learning, forgotten knowledge, fear and misconceptions become monsters.

Players are Seekers who repair damaged worlds by learning, recalling, practising and mastering subjects.

Poor marks do not make a student a villain.

Failure must be treated as feedback and another opportunity to practise.

11. WORLDS

There are four worlds:

1. The Overworld

2. The Otherworld
3. The End
4. Limbo

Each world contains:

- Four monsters
- One boss
- Story chapters
- Battle rewards
- Subject encounters
- Difficulty scaling
- World progress
- Original cards

Limbo requires:

- Player level 10
- Three owned Legendary card instances
- Seven Abstracted Errors

Server validation is mandatory.

Cards borrowed, temporarily assigned, already burned, listed for sale or locked in an incomplete trade must not count incorrectly.

12. PLAYER STATS

Default battle statistics:

- Maximum HP: 100
- Maximum Mana: 100
- Battle deck size: 5 cards
- Normal ability Mana cost: 20
- Abstracted ability Mana cost: 40

Use configuration tables rather than scattering constants through code.

Player progression includes:

- Player XP
- Level
- STP
- SLC
- Battle rating
- Study streak

- Puzzle streaks
- World progress
- Event progress
- Inventory
- Vault
- Deck

Player XP and Event EXP are separate.

13. BATTLE ENGINE

The server is authoritative.

The browser may display predictions but does not decide outcomes.

Use a deterministic engine with an injectable random seed.

Store a complete replayable battle log.

13.1 Turn Flow

Default flow:

1. Start-of-turn effects
2. Shield processing
3. Status processing
4. Optional study challenge
5. Action selection
6. Server validation
7. Mana payment
8. Target resolution
9. Damage calculation
10. Status application
11. Defeat check
12. Reward check
13. Battle log
14. End-of-turn effects
15. Next turn

13.2 Five-Card Deck

Players use five cards.

Restricted ability rules:

- Only one Poison card
- Only one Decay card
- Only one Shield card
- Only one card for each other restricted ability key

Validate deck rules:

- On deck save
- Before matchmaking
- At battle creation
- When card definitions change

Decks containing removed or invalid cards must be marked invalid and repaired before ranked play.

13.3 Shields

A valid shield card activates at the beginning of the match.

The shield lasts for the first two turns.

After turn two, it expires unless a legal ability renews it.

Only one shield card may be equipped.

13.4 Mana Recovery Quiz

A player may answer five quiz questions to restore Mana.

Default:

- Each correct answer restores 4 Mana.
- Maximum restoration is 20 Mana.
- Mana cannot exceed 100.
- Questions come from a selected subject.
- During enforced exam periods, they come from exam portions.
- Questions may not repeat within the same battle.

The game may support an all-or-nothing mode through configuration, but the default should reward each correct answer to avoid making one mistake waste the entire study attempt.

13.5 One-Turn Damage Challenge

Before an attack, the player may attempt five questions.

Rules:

- All five must be correct.
- Success grants +10 immediate damage.
- Bonus lasts for that turn only.
- It does not affect later attacks.
- One wrong answer causes the challenge to fail.
- Failure leaves normal damage unchanged.
- The challenge must be repeated for another turn.

13.6 Stackable Abilities

Some abilities add to the basic attack.

Example:

- Basic attack: 10
- Poison addition: 10
- Successful quiz boost: 10
- Immediate total: 30

Only one stackable card ability may be added to one attack unless an explicitly tested card says otherwise.

Damage-over-time must be logged separately from immediate damage.

13.7 Ability System

Abilities are data-driven.

Each ability contains:

- Key
- Name
- Category
- Description
- Mana cost
- Damage
- Healing
- Duration
- Cooldown
- Target
- Stack policy
- Status effect
- Trigger
- Restrictions
- Balance version

Categories may include:

- Attack
- Poison
- Decay
- Burn
- Bleed
- Shield
- Heal
- Restore Mana
- Drain Mana
- Cleanse
- Reflect
- Dodge
- Silence
- Stun
- Resistance
- Buff
- Debuff
- Abstracted

Avoid switch statements containing every card by name.

14. CURRENCIES

14.1 STP

StudyPoints are abbreviated STP.

STP is earned through meaningful study and gameplay.

STP may be used for:

- Loot boxes
- Marketplace purchases
- Trading fees
- Quest rerolls
- Utility items
- SLC conversion

14.2 SLC

Conversion:

- 500 STP = 100 SLC

SLC is internal fictional currency.

SLC cannot be purchased with real money.

Use integer arithmetic.

No floating-point currency.

14.3 Wallet Ledger

Every balance mutation must produce an immutable ledger entry containing:

- Transaction UUID
- User UUID
- Currency
- Amount
- Balance before
- Balance after
- Transaction type
- Reason
- Related entity
- Idempotency key
- Timestamp
- Actor
- Audit metadata

Never directly update wallet balances without the economy service.

Use database locking.

Prevent negative balances.

15. EXAM REWARDS

For an exam marked out of 60:

- 60/60: 800 STP and one Legendary card
- 50/60 or higher: 400 STP and one Rare card

Generalise thresholds:

- 100%: perfect tier
- At least 83.33% but below 100%: high tier

Require verification through:

- Teacher approval
- Administrator approval
- Trusted school import
- Result evidence review

The same exam cannot be rewarded twice.

Lower marks still contribute to ordinary study progress and improvement analytics.

Do not shame students who do not reach a special tier.

16. CARD MODEL

Separate card definitions from owned card instances.

16.1 Card Definition

A definition contains:

- Definition UUID
- Card number
- Name
- Slug
- Lore
- Rarity
- Variant
- Abilities
- Base damage
- Artwork
- Release event
- Original print supply
- Active supply
- Burned count
- Scraped count
- Official value
- Tradable
- Burnable
- Scrapable
- Event-only
- Active
- Extinct
- Balance version
- Created date

- Retired date

16.2 Card Instance

An owned copy contains:

- Instance UUID
- Definition UUID
- Print-run UUID
- Serial number
- Current owner
- Original owner
- Acquired source
- Acquired date
- Inventory location
- Trade lock
- Event lock
- Burn state
- Scrape state
- Previous-owner count

Display a shortened public serial while retaining the complete UUID internally.

16.3 Rarities

- Common
- Uncommon
- Rare
- Epic
- Legendary
- Mythic

Abstracted is a variant or state.

Examples:

- Rare Abstracted
- Epic Abstracted
- Legendary Abstracted

Normal cards usually have two abilities.

Abstracted cards have three.

17. ACTIVE CARD CAP

There may be no more than 100 active card definitions.

Extinct historical definitions remain stored but do not count toward the active cap.

The database must show:

- Original supply
- Current supply
- Burned
- Scraped
- Inventory copies
- Vault copies
- Listed copies
- Locked copies
- Extinct status
- Return eligibility
- Price history

Do not expose private owner identities.

18. INVENTORY AND VAULT

Every player has:

- Inventory capacity: 50 cards
- Vault capacity: 50 cards
- Active deck capacity: 5 cards

Vault cards:

- Cannot be used in battle.
- Cannot be listed directly.
- Must be moved into inventory first.
- Still count as owned unless a rule explicitly excludes them.

When storage is full:

- New card rewards enter a pending claim queue.
- The pending queue is temporary.
- Default expiry is configurable.
- Warn the player repeatedly.
- Never silently erase high-rarity or event rewards.
- Record expired claims.

- Allow restoration after confirmed system errors.
-

19. LOOT BOXES

There is no direct shop selling individual normal cards.

Cards enter through:

- Loot boxes
- Events
- Quests
- Exams
- Battles
- StudyPass
- Puzzles
- Trading
- Return events

All odds are server-side and published.

Use cryptographically secure random generation.

Store every opening.

19.1 Normal Loot Box

Cost:

- 100 STP

Weights:

- 10 Common
- 5 Uncommon
- 1 Rare

Probabilities:

- Common: 62.5%
- Uncommon: 31.25%
- Rare: 6.25%

One card is awarded per opening.

19.2 Boosted Loot Box

Cost:

- 200 STP

Weights:

- 5 Common
- 5 Uncommon
- 3 Rare

Probabilities:

- Common: approximately 38.46%
- Uncommon: approximately 38.46%
- Rare: approximately 23.08%

One card is awarded per opening.

19.3 Event Loot Boxes

Events may separately include:

- Epic
- Legendary
- Mythic
- Abstracted
- Returning extinct cards

Not every event must contain every rarity.

Event odds must be versioned and published.

Do not secretly change odds during an event.

No loot box may use real money.

Avoid casino-like animations, near-miss manipulation and dark patterns.

20. MARKETPLACE AND TRADING

Support:

- Fixed-price listings

- Direct trade
- Offers
- Expiry
- Cancellation
- Price history
- Trade receipts
- Reporting
- Moderation

Use PostgreSQL transactions and row locks.

Before settlement, verify:

- Seller still owns the card.
- Card is tradable.
- Card is not burned.
- Card is not scraped.
- Card is not in another trade.
- Card is not in an active deck.
- Buyer has enough currency.
- Buyer has storage capacity or pending-claim eligibility.

Event reward cards may be marked untradeable.

Never trust frontend ownership state.

21. CARD VALUE

Maintain an official reference price.

It may consider:

- Base rarity value
- Original supply
- Current supply
- Removal rate
- Recent completed sales
- Recent demand
- Battle usage
- Event demand

Use bounded multipliers.

Prevent sudden extreme changes.

Ignore obvious self-trading and suspicious circular trades.

Store historical values.

Scraper and Burner payments use the official value at confirmation, not a player's listing price.

22. CARD SCRAPER

The Card Scraper:

- Permanently removes a card instance.
- Pays its official liquidation value immediately.
- Does not normally grant burn-event progress.
- Updates supply.
- Creates a wallet entry.
- Creates a supply-ledger entry.
- Checks for extinction.
- Cannot be reversed by the user.

Require confirmation showing:

- Card
 - Serial
 - Rarity
 - Current supply
 - Refund
 - Permanent-removal warning
-

23. CARD BURNER

The Card Burner:

- Permanently removes a card.
- Pays through instalments.
- May grant Event EXP.
- May grant quest progress.
- May grant a tradeable event item.
- Creates a burn receipt.

Default instalments:

- Four equal or near-equal payments
- First payment immediately
- Three later daily payments
- Total fixed at burn confirmation

- Final instalment handles integer remainder

Use idempotent BullMQ jobs.

A payment must never be duplicated.

24. EXTINCTION

When supply reaches zero:

1. Lock the definition.
2. Confirm no active instances remain.
3. Mark it extinct.
4. Remove it from loot pools.
5. Preserve history.
6. Retire its print run.
7. Activate or create a replacement.
8. Give the replacement a new definition UUID.
9. Create a new print run.
10. Announce the extinction.
11. Invalidate caches.
12. Update the active-card count.

Burned UUIDs never return.

Extinct definitions may return in a special event, but returning copies receive new instance UUIDs and a new print run.

25. EVENTS

One event must always be active.

Events include:

- Name
- Story
- Start
- End
- Grace period
- Claim deadline
- Quests
- StudyPass
- Event items

- Battles
- Cards
- Loot boxes
- Extinction targets
- Return cards
- Shop configuration

If no scheduled event follows an ending event, automatically activate a safe fallback event called:

Study Sprint

Administrators must receive warnings before this occurs.

Use distributed locks so two processes cannot start the same event twice.

26. STUDYPASS

Each normal event has 14 levels.

Cumulative EXP:

1. 0
2. 100
3. 200
4. 300
5. 400
6. 550
7. 700
8. 900
9. 1100
10. 1300
11. 1450
12. 1550
13. 1650
14. 1750

Event EXP is separate from player XP.

27. FREE AND GOLD TRACKS

Each event has:

- Free Pass

- Gold Pass

Gold Pass cost:

- 1500 SLC per event

Rules:

- Gold Pass is purchased separately for every event.
 - A player uses only one track.
 - Gold players cannot claim Free rewards.
 - Free players cannot claim Gold rewards.
 - The choice locks before the first reward claim.
 - No double claiming.
 - Gold Pass cannot be bought with real money.
 - Gold rewards may be more reliable but must not guarantee competitive dominance.
-

28. ABSTRACTED EVENT

The first event is:

Abstracted

Abstracted cards:

- Have three abilities.
- Include one Abstracted ability.
- Use 40 Mana for the Abstracted ability.
- Are unstable.
- React to defined battle conditions.
- Use original visual design and lore.

28.1 Unabstracting

Unabstracting an eligible card:

- Is irreversible.
- Converts or retires the Abstracted instance according to its definition.
- Produces a configured Legendary result.
- Grants one Abstracted Error.
- Grants 500 STP.
- Updates supply.
- Creates economy and card audit entries.
- Requires explicit confirmation.

Seven Abstracted Errors are required for Limbo.

28.2 Abstracted Free Track

1. 0 EXP — 100 STP
2. 100 EXP — Normal Loot Box
3. 200 EXP — 75 STP
4. 300 EXP — Common Loot Box
5. 400 EXP — 125 STP
6. 550 EXP — Uncommon+ Loot Box
7. 700 EXP — 200 STP
8. 900 EXP — Rare Loot Box
9. 1100 EXP — Normal Loot Box and 100 STP
10. 1300 EXP — Abstracted Fragment
11. 1450 EXP — Untradeable Rare event card
12. 1550 EXP — 300 STP
13. 1650 EXP — Epic-Chance Event Loot Box
14. 1750 EXP — Untradeable Legendary Abstracted card

28.3 Abstracted Gold Track

1. 0 EXP — 200 STP
2. 100 EXP — Boosted Loot Box
3. 200 EXP — Event Loot Box
4. 300 EXP — 200 STP
5. 400 EXP — Guaranteed Rare card
6. 550 EXP — Rare Loot Box
7. 700 EXP — 300 STP
8. 900 EXP — Guaranteed Epic event card
9. 1100 EXP — Boosted Loot Box and 150 STP
10. 1300 EXP — Epic-Chance Event Loot Box
11. 1450 EXP — Untradeable Rare event card and 200 STP
12. 1550 EXP — 500 STP
13. 1650 EXP — Legendary-Chance Event Loot Box
14. 1750 EXP — Untradeable Legendary Abstracted card

“Legendary-Chance” does not mean guaranteed.

Publish its exact odds.

29. GREAT EXTINCTION EVENT

The next major event is:

The Great Extinction

It targets exactly ten active card definitions.

Default composition:

- Five Common-to-Rare cards
- Five weaker or underused Legendary cards

Future events may target a Mythic, but this requires:

- Explicit warning
- Second confirmation
- Reason
- Audit entry
- Supply simulation
- Event-balance review

29.1 Optional Burning

Burning must be strongly encouraged but not literally unavoidable.

Create a tradeable event item:

Extinction Sigil

A Sigil may be earned by:

- Burning a targeted card
- Completing a difficult alternative study quest
- Completing a global milestone
- Trading with another player

A Sigil can satisfy selected event objectives.

This allows a player to preserve a valued card.

Final event reward cards remain untradeable.

30. QUESTS

Quest categories:

- Daily
- Weekly
- Study
- Puzzle

- Battle
- Exam
- Event
- Story
- Burn
- Marketplace
- Community

Conditions must be data-driven.

Rewards may contain:

- STP
- SLC
- Player XP
- Event EXP
- Cards
- Loot boxes
- Event items
- Puzzle streak shields
- Utility items

Examples:

- Study Mathematics for 20 verified minutes.
- Answer 15 Science questions.
- Review 20 due flashcards.
- Correct three mistakes from the mistake notebook.
- Solve three ranked puzzles.
- Restore Mana in battle.
- Complete a successful damage challenge.
- Defeat an Overworld monster.
- Burn a targeted card.
- Obtain an Extinction Sigil.
- Complete a Kaushal Vikas programme task.

Quest progress must be idempotent.

31. LEADERBOARDS

Categories:

- Player XP
- Study time
- Study streak
- Quiz accuracy

- Questions answered
- Puzzle streak by subject
- World progress
- Ranked battle rating
- Event EXP
- Exam achievements
- Cards discovered
- Total STP earned

Do not use current wallet balance as the main economy leaderboard.

Support:

- Weekly
- Monthly
- Event
- All-time
- Classroom
- Friends where applicable

Use Redis sorted sets for fast current rankings.

Persist authoritative ranking events and season results in PostgreSQL.

Allow privacy opt-out.

32. SECURITY AND PRIVACY

The users may be minors.

Implement:

- Secure password hashing
- Refresh-token rotation
- Session revocation
- Rate limiting
- CSRF protection where applicable
- Strict CORS
- Security headers
- Content Security Policy
- Input validation
- Output sanitisation
- File validation
- Resource ownership checks
- Role checks

- Audit logs
- Account export
- Account deletion
- Minimal public profiles
- Moderation and reporting
- No public email addresses
- No unrestricted private messaging

Never expose:

- Database credentials
- Redis credentials
- Qdrant keys
- AI keys
- Storage keys
- JWT secrets
- Internal card instance history
- Private documents
- Correct quiz answers before submission

Qdrant queries must always include mandatory tenant or ownership filters.

Test cross-user data isolation.

33. FAILURE MODES

Design explicitly for failure.

PostgreSQL Unavailable

- Readiness check fails.
- Mutating routes return a safe service-unavailable response.
- Do not accept economy or trade operations.
- Do not fabricate success.

Redis Unavailable

- Do not corrupt authoritative data.
- Pause queue-dependent features.
- Fall back to local cache only for safe public reads.
- Rate-limited endpoints fail safely.
- Battle matchmaking pauses.
- Show degraded status.
- BullMQ HTTP submissions should fail quickly rather than hang indefinitely.

Qdrant Unavailable

- Uploaded documents remain safely stored.
- Ingestion jobs retry.
- Exact PostgreSQL search may remain available.
- Do not generate a supposedly grounded answer without retrieved evidence.
- Display RAG temporarily unavailable.

AI Provider Unavailable

- Manual study tools continue working.
- RAG retrieval can show relevant source passages.
- AI generation displays a clear error.
- Jobs retry within limits.
- Do not endlessly retry.

Worker Unavailable

- API remains available.
- Outbox events remain pending.
- Admin status page reports queue delay.
- Restarted workers safely resume idempotent jobs.

Storage Unavailable

- Reject new uploads safely.
- Existing metadata remains intact.
- Do not create a ready document without its file.

34. PERFORMANCE TARGETS

Optimise based on measurements.

Initial goals under normal load:

- Cached public API p95: below 150 ms
- Ordinary authenticated read p95: below 300 ms
- Ordinary mutation p95: below 500 ms
- Battle turn resolution excluding quiz answering: below 500 ms
- Initial dashboard useful render on mid-range mobile: below 3 seconds on a reasonable connection
- No initial JavaScript bundle unnecessarily containing the full Study RPG and all Studyield tools
- Search result response before generation: below 1.5 seconds under normal starter load
- No unbounded database query
- No unbounded Redis key

- No unbounded queue history

Create load tests for:

- Login
- Dashboard
- Quizzes
- Loot boxes
- Battle turns
- Marketplace
- Leaderboards
- RAG retrieval
- Event reward claims

Use indexes based on query plans, not guesswork.

35. DATABASE SAFETY

Use:

- UUIDs
- UTC timestamps
- Foreign keys
- Check constraints
- Unique constraints
- Partial indexes
- Transaction isolation appropriate to each operation
- Row locks for economy and card settlement
- Soft deletion where audit history is required
- Immutable ledgers
- Migration checksums

Prevent through database constraints where practical:

- Negative balances
- Duplicate reward claims
- More than five deck cards
- Duplicate card instance in a deck
- Multiple active listings for one card
- Trading untradeable cards
- Owning a burned card
- Duplicate burn payment
- Claiming both pass tracks
- Active supply below zero
- Duplicate exam reward
- Duplicate loot opening settlement

36. OBSERVABILITY

Implement:

- `/health/live`
- `/health/ready`
- `/health`
- `/version`
- Request IDs
- Job IDs
- Battle IDs
- Trade IDs
- Structured logs
- Queue metrics
- Redis metrics
- PostgreSQL pool metrics
- Qdrant latency
- RAG retrieval diagnostics
- Worker heartbeat
- Error tracking adapter
- Admin status dashboard

Do not log:

- Passwords
- Tokens
- API keys
- Full private documents
- Unredacted sensitive user data

Create runbooks for:

- Redis outage
 - PostgreSQL outage
 - Qdrant outage
 - Stuck queue
 - Failed ingestion
 - Failed burn instalment
 - Duplicate-reward incident
 - Marketplace dispute
 - Event-start failure
 - Restore from backup
-

37. LOCAL DEVELOPMENT

Provide Docker Compose with:

- PostgreSQL
- Redis
- Qdrant
- Optional ClickHouse profile
- Optional MinIO
- Optional Mailpit
- Backend API
- Worker
- Frontend

Do not use `latest` image tags in production-oriented configuration.

Pin compatible versions.

Add health checks.

Add persistent volumes.

Provide seed commands.

A developer should be able to run the full platform locally with documented commands.

38. CLOUDFLARE PAGES DEPLOYMENT

Deploy the frontend from its monorepo directory.

Document:

- Root directory
- Build command
- Output directory
- Environment variables
- Preview deployment
- Production branch
- Build watch paths
- SPA behaviour
- Security headers
- Custom domain
- Rollback

Expected Vite settings:

- Build command equivalent to `pnpm build`
- Output directory `dist`

Create a `_headers` file with a tested Content Security Policy and other security headers.

Do not add custom asset caching that causes stale builds without evidence.

No secret may be stored in a `VITE_` variable.

Only public values such as API base URL may use frontend environment variables.

39. ENZONIC DEPLOYMENT

Prepare separate persistent runtime processes for:

- API
- Worker
- PostgreSQL
- Redis

Use Qdrant through either:

- A separately managed Qdrant deployment
- A properly secured self-hosted deployment
- A configured Qdrant Cloud cluster

Do not expose PostgreSQL, Redis or Qdrant publicly without authentication and network restrictions.

Document:

- Build command
- Start command
- Worker command
- Port
- Environment variables
- Health endpoint
- Persistent directories
- Database migrations
- Redis configuration
- Qdrant configuration
- Storage configuration
- WebSocket proxying
- Graceful shutdown

- Restart policy
- Backup
- Restore

The API and worker must shut down gracefully.

Stop accepting new work before terminating.

Close:

- HTTP server
 - WebSockets
 - Database pool
 - Redis clients
 - BullMQ workers
 - Qdrant client activity
-

40. BACKUPS

Create a backup strategy for:

- PostgreSQL
- Uploaded files
- Qdrant snapshots
- Redis persistence files where useful
- Environment configuration documentation

PostgreSQL and uploaded files are the most important.

Test restoration.

A backup that has never been restored in a test is not considered verified.

41. TESTING GATES

Required automated tests:

Core

- Auth
- Permissions
- Data isolation
- Migrations

- Validation
- Rate limiting
- Queue jobs
- Outbox dispatcher

Study Tools

- Task rewards
- Focus sessions
- Flashcard scheduling
- Quiz scoring
- Puzzle streaks
- Exam portions
- Mistake notebook
- Offline sync conflicts

RAG

- Document parsing
- Chunking
- Deduplication
- Access filtering
- Hybrid retrieval
- Citation mapping
- Reindexing
- Deletion
- Cross-user leakage
- Insufficient-evidence response

Game

- Five-card deck
- Restricted abilities
- Mana
- Shields
- Damage challenge
- Status effects
- Victory
- Rewards
- Replay determinism

Economy

- Currency conversion
- Wallet locking
- Loot odds
- Duplicate opening request

- Marketplace race
- Trade settlement
- Scraper
- Burner
- Instalments
- Extinction
- Replacement
- Inventory overflow

Events

- Event scheduler
- Always-active fallback
- Free and Gold track lock
- Level 14 at 1750 EXP
- Untradeable event cards
- Abstracted Errors
- Limbo requirements
- Extinction Sigils

End-to-End

Test actual user flows:

1. Register.
2. Create a subject.
3. Create a study task.
4. Complete a focus session.
5. Take a quiz.
6. Earn STP.
7. Open a loot box.
8. Receive a unique card.
9. Create a deck.
10. Fight a monster.
11. Restore Mana.
12. Complete a damage challenge.
13. Move a card to the Vault.
14. List a card.
15. Trade a card.
16. Burn a card.
17. Receive an instalment.
18. Upload a document.
19. Wait for indexing.
20. Ask a grounded question.
21. Receive correct citations.
22. Progress an event quest.
23. Claim a StudyPass reward.

42. IMPLEMENTATION PHASES

Phase 0 — Audit

- Repository map
- Build failures
- Test failures
- Security findings
- Licence contradiction
- Feature inventory
- Dependency inventory
- Database inventory
- Deployment inventory

Phase 1 — Architecture Stabilisation

- Workspace setup
- Configuration validation
- PostgreSQL
- Redis
- BullMQ
- Qdrant
- API and worker split
- Health checks
- Logging
- Outbox
- Authentication
- Licence notices

Phase 2 — Studyield Core

- Academic structure
- Dashboard
- Tasks
- Focus
- Notes
- Flashcards
- Quizzes
- Mistake notebook
- Puzzles
- Exam periods
- Analytics

Phase 3 — RAG

- Storage
- Uploads
- Parsing
- Chunking
- Embeddings
- Qdrant
- Hybrid search
- Citations
- RAG chat
- Evaluation

Phase 4 — Game Foundation

- Player profile
- XP
- STP
- SLC
- Wallet ledger
- Cards
- Inventory
- Vault
- Deck
- Loot boxes

Phase 5 — Battles

- Worlds
- Monsters
- Bosses
- Battle engine
- Mana quizzes
- Damage challenges
- Shields
- Status effects
- Rewards
- PvP

Phase 6 — Economy

- Marketplace
- Trades
- Pricing
- Scraper
- Burner
- Instalments

- Supply ledger
- Extinction
- Replacement

Phase 7 — Events

- Scheduler
- Quests
- StudyPass
- Free and Gold tracks
- Abstracted
- Unabstracting
- Great Extinction
- Extinction Sigils

Phase 8 — Advanced Learning

- Exam Clone
- Problem Solver
- Teach-Back
- Learning Paths
- Formula tools
- Concept tools
- Optional research
- Optional code sandbox after security review

Phase 9 — Hardening

- Performance
- Accessibility
- Offline mode
- Security
- Privacy
- Load testing
- Backups
- Deployment
- Documentation

Do not begin a later phase while the previous phase fails its build and tests.

43. DEFINITION OF DONE

The rebuild is complete only when:

- Frontend installs and builds.

- Backend installs and builds.
- Worker installs and starts.
- PostgreSQL migrations work from a clean database.
- Redis is connected and health checked.
- BullMQ processes jobs.
- Qdrant ingestion and retrieval work.
- RAG answers contain verified citations.
- Cross-user RAG access is impossible in tests.
- Study tools work without AI.
- Study activity grants controlled rewards.
- The card system uses definition and instance UUIDs.
- Inventory and Vault limits work.
- Loot odds match published weights.
- Deck restrictions work.
- Battles are server-authoritative.
- Mana recovery works.
- One-turn damage boost works.
- Shields last two turns.
- Marketplace settlement is transactional.
- Scraping is permanent.
- Burning pays idempotent instalments.
- Extinction replaces cards safely.
- One event is always active.
- StudyPass has 14 levels.
- Level 14 requires 1750 EXP.
- Free and Gold tracks are mutually exclusive.
- Abstracted final rewards work.
- Limbo requirements work.
- Redis outage does not corrupt permanent data.
- Qdrant outage does not create false grounded answers.
- Cloudflare deployment configuration is documented.
- Enzonic deployment configuration is documented.
- Backups and restoration are documented.
- Licence contradiction is explicitly handled.
- No core route is a placeholder.
- No test result is falsely reported.

44. FINAL CODEX REPORT

At the end, provide:

1. Repository audit summary
2. Licence findings
3. Existing features retained
4. Existing features removed

5. Existing features rewritten
6. New modules
7. New migrations
8. Redis architecture
9. BullMQ queues
10. Qdrant collection design
11. RAG evaluation results
12. Battle-engine test results
13. Economy test results
14. Security findings
15. Remaining risks
16. Frontend build result
17. Backend build result
18. Worker startup result
19. Test commands run
20. Exact test outcomes
21. Local startup commands
22. Cloudflare Pages configuration
23. Enzonic API configuration
24. Enzonic worker configuration
25. PostgreSQL configuration
26. Redis configuration
27. Qdrant configuration
28. Required environment variables
29. Backup and restoration procedure
30. Manual steps requiring account credentials

Do not claim deployment succeeded unless it actually succeeded.

Do not claim tests passed unless they were executed.

45. BEGINNING INSTRUCTION

Begin now.

First:

1. Inspect the complete repository.
2. Run the current install, tests and builds.
3. Create the repository audit.
4. Create the dependency and infrastructure decision matrix.
5. Investigate the licence contradiction.
6. Create `AGENTS.md`.
7. Create `IMPLEMENTATION_STATUS.md`.
8. Create the phased Goose recipes using the installed Goose schema.

9. Stabilise PostgreSQL, Redis, BullMQ and Qdrant before implementing new game features.
10. Continue phase by phase until every Definition of Done item is either completed or honestly documented as blocked.

Do not respond with only a plan.

Start inspecting and implementing.